

第9章 树

张猛

华中科技大学计算机科学与技术学院
新型非易失存储系统实验室 (NNSS)

zgmeng@hust.edu.cn

2026年6月23日

Section 9.1

树的概述

内容

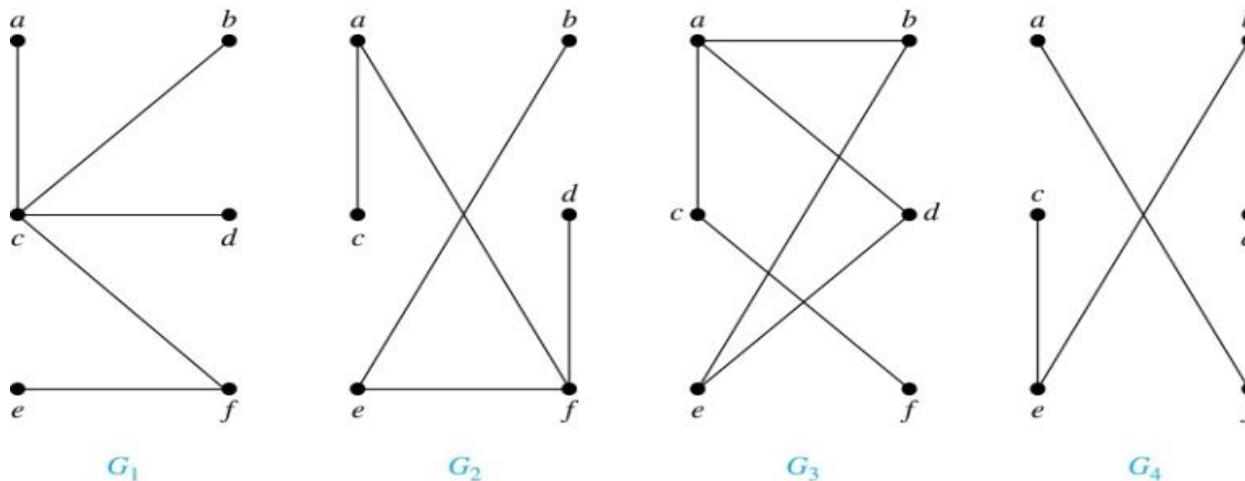
- 有根树
- 树作为模型
- 树的性质

树

- 定义: **树** 是没有简单回路的连通无向图.
- 因为树没有简单回路, 那么树不包含多重边或环. 树必然是简单图.

树

□例:如图所示, 哪些是树?

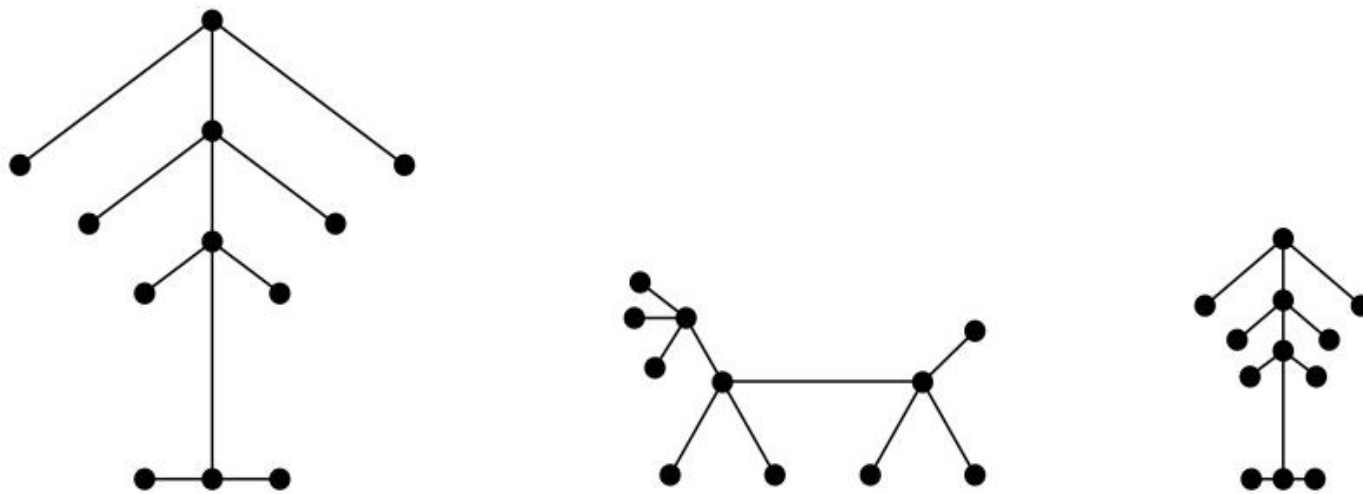


□解:

- G_1 和 G_2 是树, 因为都没有简单回路的连通图.
- G_3 不是树, 因为 e, b, a, d, e 是该图中的一条简单回路.
- G_4 不是树, 因为它不连通.

树

□ 定义:**森林**是指不包含简单回路, 但不一定连通的图. 森林中每个连通分支都是树.



如图所示是一个具有3个连通分支的森林

树

□定理:一个无向图是树当且仅当在它的每对顶点之间存在唯一的简单通路.

□证:

- (树的基本定义: 树是没有简单回路的连通无向图) 设 T 是树, 则 T 没有简单回路的连通图. 设 x 和 y 是 T 的两个顶点. 因为连通, 所以 x 和 y 之间存在一条简单通路. 而且这条简单通路是唯一的. 因为如果存在第二条这样的简单通路, 那么将第一条简单通路和第二条简单通路逆转后得到从 y 到 x 的通路, 这会形成回路. 这蕴含了在 T 中存在简单回路. 因此在树的任何两个顶点之间存在唯一的简单通路.

树

□定理:一个无向图是树当且仅当在它的每对顶点之间存在唯一的简单通路.

□证(续):

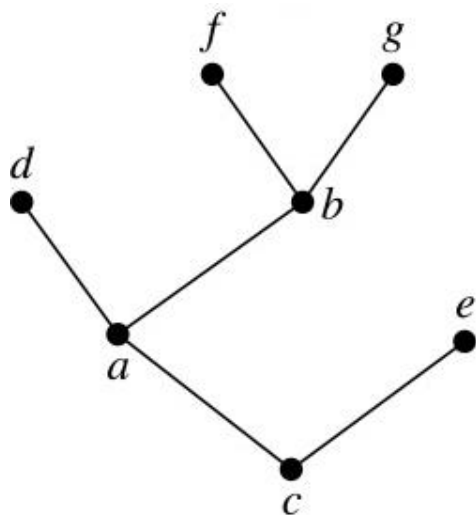
- (树的基本定义: 树是没有简单回路的连通无向图) 现在假定图 T 中任何两个顶点之间存在唯一的简单通路. 则 T 是连通的, 因为在它的任何两个顶点之间存在通路. 另外 T 没有简单回路. 反证: 假定 T 有包含顶点 x 和 y 的简单回路, 则在 x 和 y 之间就有两条简单通路, 因为这条简单回路包含一条从 x 到 y 的简单通路和一条从 y 到 x 的简单通路. 因此, 在任何两个顶点之间存在唯一的简单通路的图是树.

有根树

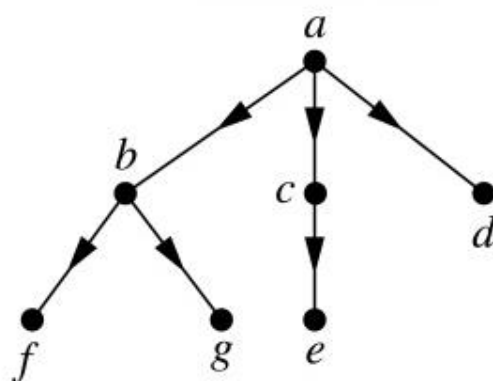
- 在树的应用中, 指定树的一个顶点作为根. 一旦规定了根, 就可以给每条边指定方向. 因为根据定理从根到图中每个顶点都存在唯一通路, 所以指定每条边是离开根的方向. 因此, 树与它的根一起产生了一个有向图, 称为有根树.
- 定义:**有根树**是指定一个顶点作为根并且每条边的方向都离开根的树.

有根树

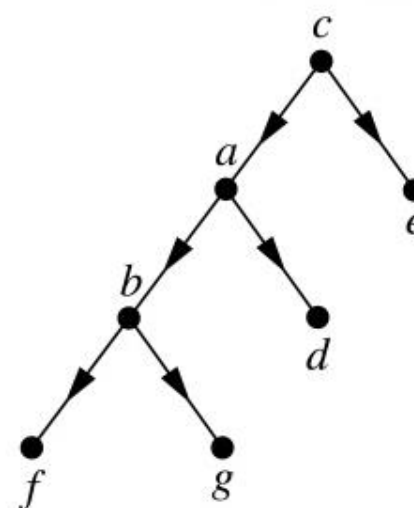
- 一个无根树可以通过选定不同的根而转换为不同的有根树。
- 例: 如图所示的树T, 指定顶点a可以形成有根树, 也可以指定顶点c形成有根树



树T



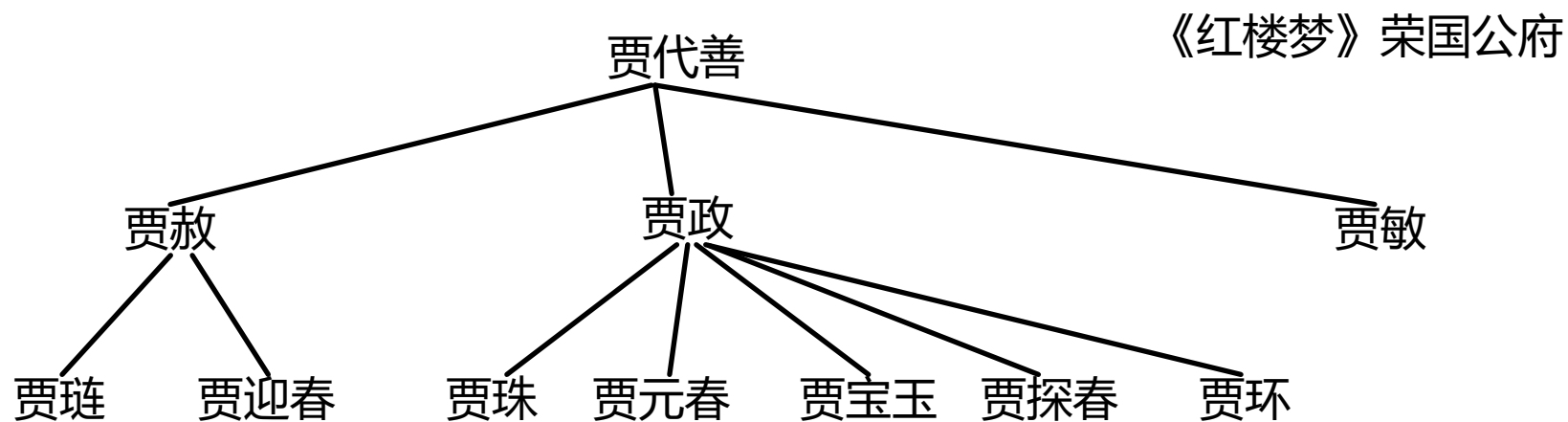
以a为根的有根树



以c为根的有根树

有根树

□ 树的术语来源于植物学和族谱学



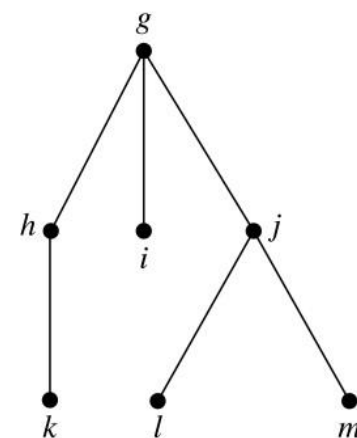
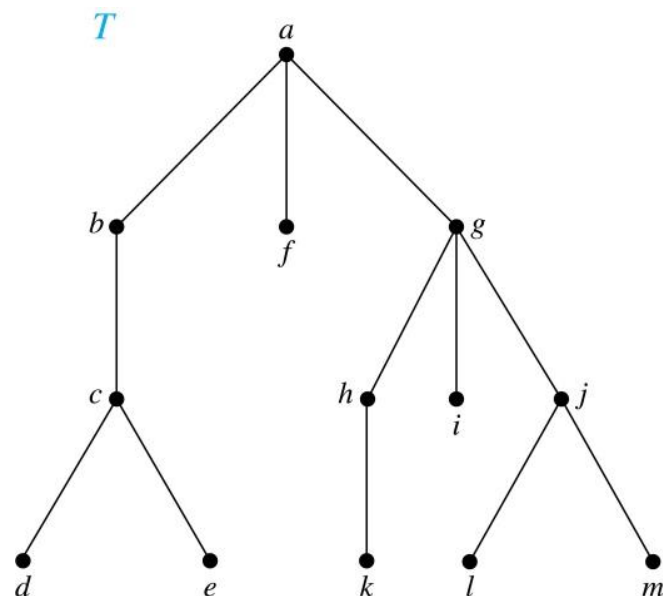
□ 设 T 是有根树, 若 v 是 T 的非根顶点, 则 v 的**父母**是从 u 到 v 存在有向边的唯一顶点 u . 当 u 是 v 的父母时, v 称为 u 的**孩子**. 具有相同父母的顶点称为**兄弟**. 树的顶点若没有孩子则称为**树叶**. 有孩子的顶点称为**内点**.

有根树

- 非根顶点的**祖先**是从根到该顶点通路上的顶点, 不包括该顶点自身但包括根(即该顶点的父母, 该顶点的父母的父母等等, 直到根). 顶点 v 的**后代**是以 v 作为祖先的顶点.
- 若 a 是树中的顶点, 则以 a 为根的**子树**是由 a 和 a 的后代以及这些顶点所关联的边所组成的该树的子图.

有根树

□例:图示的有根树中, 求c的父母, g的孩子, h的兄弟, e的所有祖先, b的所有后代, 所有内点以及所有树叶. 什么是以g为根的子树?



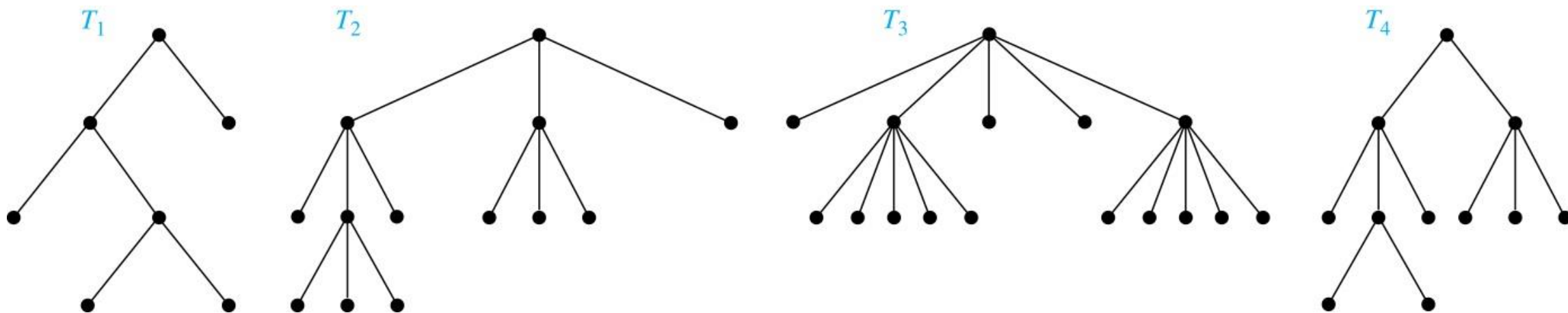
□解:c的父母是b. g的孩子是h, i, j. h的兄弟是i, j. e的祖先是c、b、a. b的后代是c、d、e. 内点是a, b, c, g, h, j. 树叶是d, e, f, i, k, l, m. 以g为根的子树如右图所示.

有根树

- 定义: 若有根树的每个内点都有不超过 m 个孩子, 则称它是 **m 叉树**.
若该树的每个内点都恰好 m 个孩子, 则称它是**满 m 叉树或 m 叉正则树**.
把 $m=2$ 的 m 叉树称为**二叉树**.
- 在有根树中顶点 v 的**层**是从根到这个顶点的唯一通路的长度. 根的层定义为0.
- 有根树的**高度**就是顶点层数的最大值. 即有根树的层数是从根到任意顶点的最长通路的长度.
- 每个树叶的层数均为树高, 则称 m 叉正则树为 **m 叉完全正则树**

有根树

□例: 如图的有根树, 对某个正整数 m 来说是否为满 m 叉树?



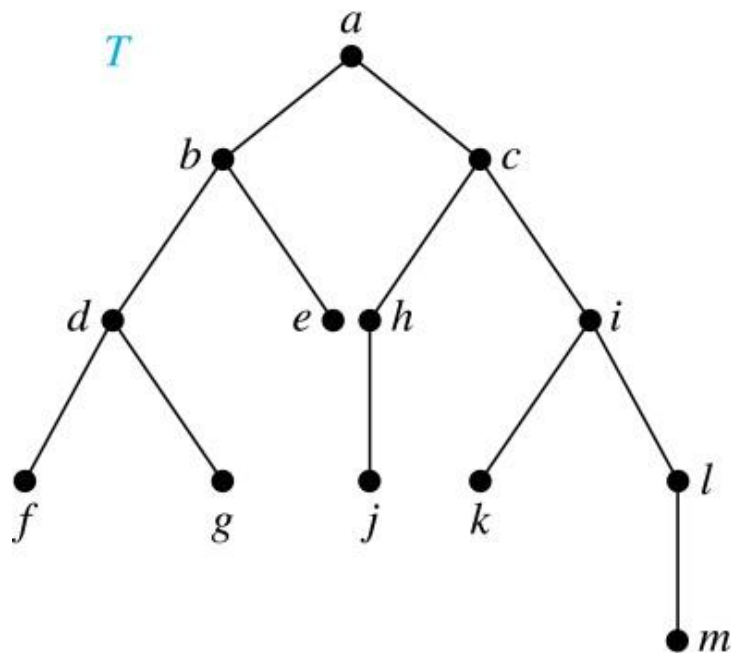
□解: T_1 是满二叉树, 因为每个内点都有 2 个孩子. T_2 是满三叉树, 因为每个内点都有 3 个孩子. T_3 是满五叉树, 因为每个内点都有 5 个孩子. T_4 不是满 m 叉树, 因为有些内点有 2 个孩子, 有些内点有 3 个孩子.

有根树

- 定义:**有序根树**是把每个内点的孩子都排序的有根树. 画有序根树时, 以从左向右的顺序来显示每个内点的孩子.
- 在有序二叉树(通常只简称为二叉树)中, 若一个内点有2个孩子, 则第一个孩子称为**左子**, 而第二个孩子称为**右子**. 以一个顶点的左子为根的树称为该顶点的**左子树**, 而以一个顶点的右子为根的树称为该顶点的**右子树**.

有根树

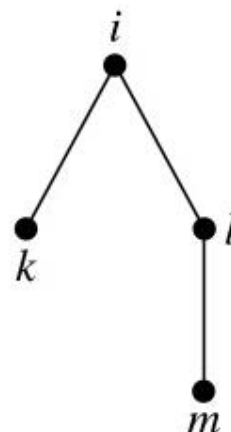
□例: 如图(a)所示的二叉树T中, d的左子和右子是什么? c的左子树和右子树是什么?



(a)



(b)



(c)

□解: d的左子是f, 右子是g. (b)和(c)分别画出来c的左子树和右子树.

树的性质

□ T 是满 m 叉树, n 是它的顶点数, i 是它的内点数, l 是它的树叶数. 那么只要知道这三个量中的任何一个, 其他两个都可以求出. 如下定理所示

□ 定理: 一个满 m 叉树若有

- (i) n 个顶点, 则有 $i = (n - 1)/m$ 个内点, $l = [(m - 1)n + 1]/m$ 个树叶
- (ii) i 个内点, 则有 $n = mi + 1$ 个顶点, $l = (m - 1)i + 1$ 个树叶
- (iii) l 个树叶, 则有 $n = (ml - 1)/(m - 1)$ 个顶点, $i = (l - 1)/(m - 1)$ 个内点

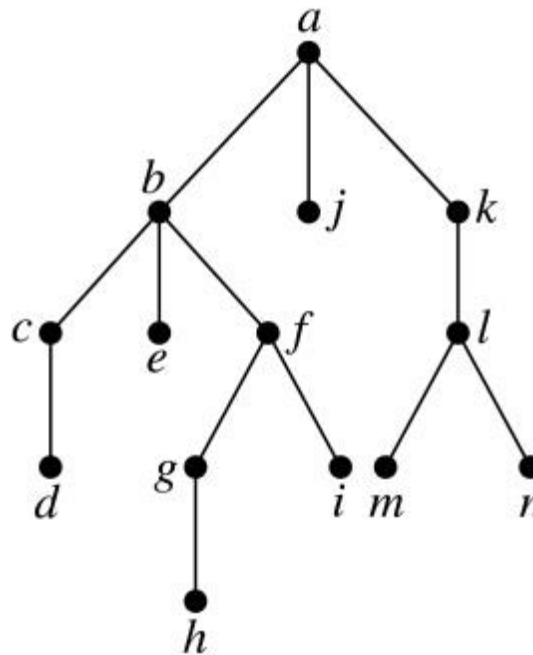
□ 证明: 根据定理 $n = mi + 1, n = i + l$ 就可以证明(i). 首先根据 $n = mi + 1$, 求解得到 $i = (n - 1)/m$. 将 i 的表达式带入 $n = i + l$, 可以得到 $l = [(m - 1)n + 1]/m$. 其他两个类似的证明.

树的性质

- 例:假设某人寄出一封连环信, 要求收到信的每个人再把它寄给另外4个人. 有些人按要求做了, 有些人则没有寄出信. 若没有人收到超过一封信, 而且若读过信但是不寄出它的人为100个后, 连环信就终止了, 那么包括第一个人在内, 有多少人看过信? 有多少人寄出过信?
- 解:4叉树 T 表示连环信. 内点对应寄出信的人, 树叶对应不寄出信的人. 因为100个不寄出信的人, 所以 T 中树叶数 $l = 100$. 根据定理可得看过信的人 $n = 133$. 内点数也就是寄出过信的人 $i = 33$.

树的性质

□例:如图所示的有根树T中每个顶点的层数是多少, T的高度是多少?

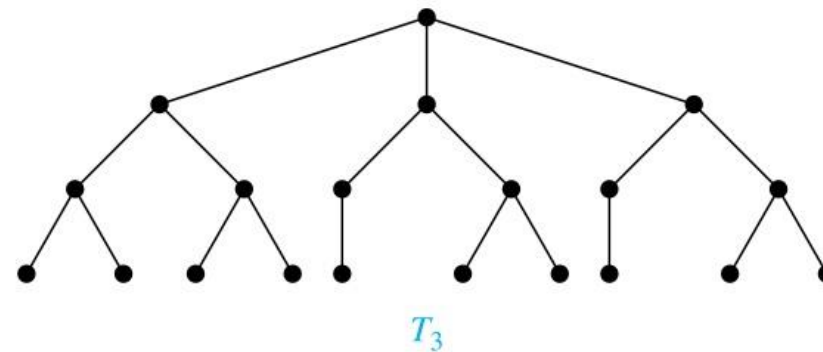
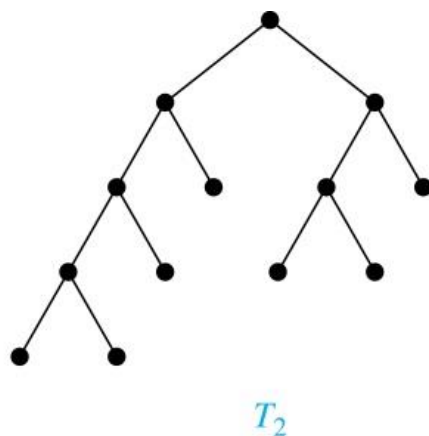
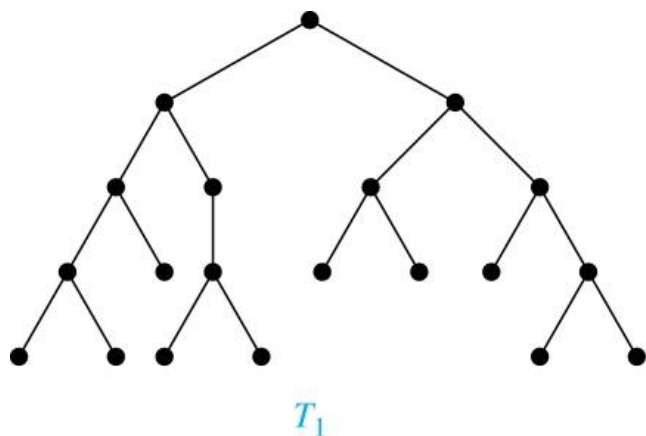


□解: a的层为0. 顶点b, j, k的层是1. c, e, f, l的层是2. d, g, i, m, n 的层是3. h 的层是4. T的高度为4, 因为任意顶点的最大层数是4.

树的性质

□ 定义: 若一棵高度为 h 的 m 叉树的所有树叶都在 h 层或 $h-1$ 层, 则这棵树是**平衡的**.

□ 例: 如果所示中哪些有根树是平衡的?



□ 解: T_1 是平衡的, 因为它的所有树叶都在3层和4层上. T_2 不是平衡的, 因为它的树叶有在2层, 3层, 4层. T_3 是平衡的, 因为它所有的树叶都在3层上.

树的性质

- **在 m 叉树中树叶数的界**, 常常会用到在 m 叉树中树叶数的上界, 它可以用 m 叉树的高度给出这样的界, 如下
- 定理: 在高度为 h 的 m 叉树中最多有 m^h 个树叶.

树的性质

- 推论1:若一棵高度为 h 的 m 叉树带有 l 个树叶, 则 $h \geq \lceil \log_m l \rceil$. 若这棵 m 叉树是满的和平衡的, 则 $h = \lceil \log_m l \rceil$.
- ($\lceil x \rceil$ 是向上取整函数, 表示大于或等于 x 的最小整数)

Section 9.2

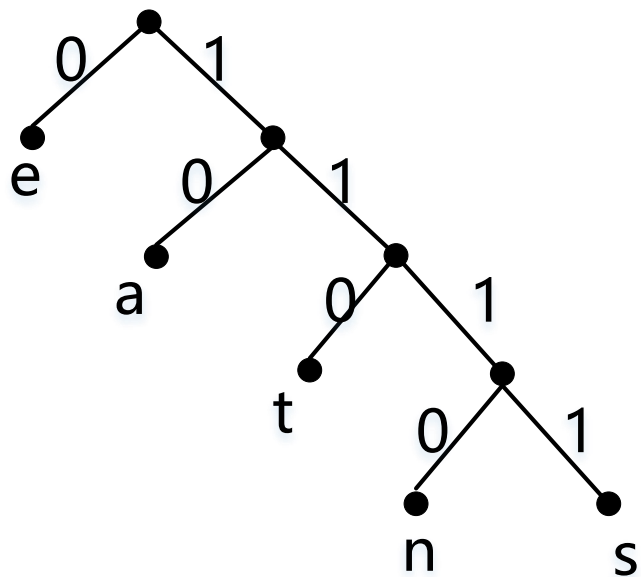
树的应用

前缀码

- ❑ 问题：用位串来编码英语字母表里的字符, 可以用长度为5的位串来表示每个字母(可以表示32种情况), 因为有26个字母. 这时每个字母都用长度为5的位串来编码, 编码数据的总长度就是5乘以文本中的字符数.
- ❑ 优化：用不同长度的位串来进行编码, 其中较短的位串来编码频繁出现的字母, 较长的位串来编码不常出现的字母.
- ❑ 为了保证没有位串对应着多个字母的序列, 可以令一个字母的位串永远不出现在另一个字母的位串的开头部分. 具有这个性质的编码称为**前缀码**.

前缀码

- 前缀码可以用二叉树来表示, 其中字符是树叶的标记. 树的边也被标记, 使得通向左子的边标记为0, 通向右子的边标记为1. 用来编码一个字符的位串就是从根到以这个字符作为标记的树叶的唯一通路上标记的序列. 如下图中s编码为1111. 编码为1111011100的单词为sane.



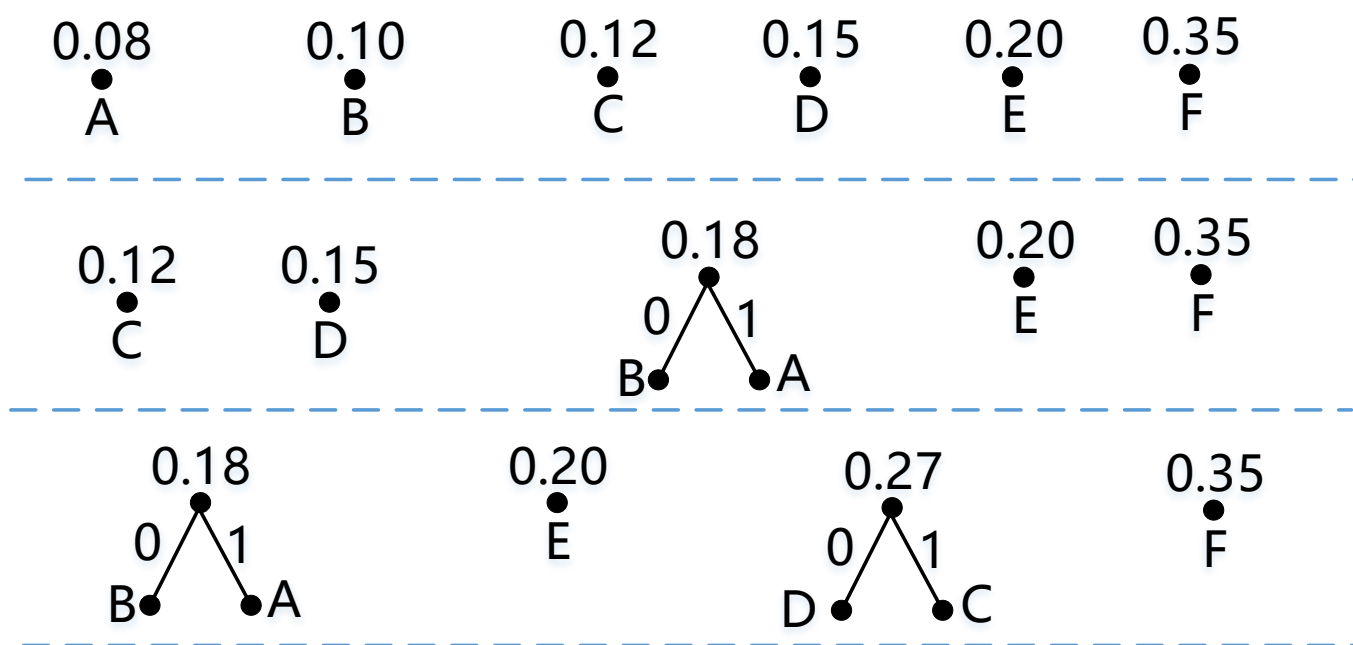
前缀码

□ **哈夫曼编码**是一种经典的前缀码. 哈夫曼编码算法用一个字符串中符号出现的频率作为输入, 并产生编码这个字符串的一个前缀码作为输出, 在这些符号的所有可能的二叉前缀码中, 哈夫曼编码使用最少的位. 它广泛用于数据压缩方面.

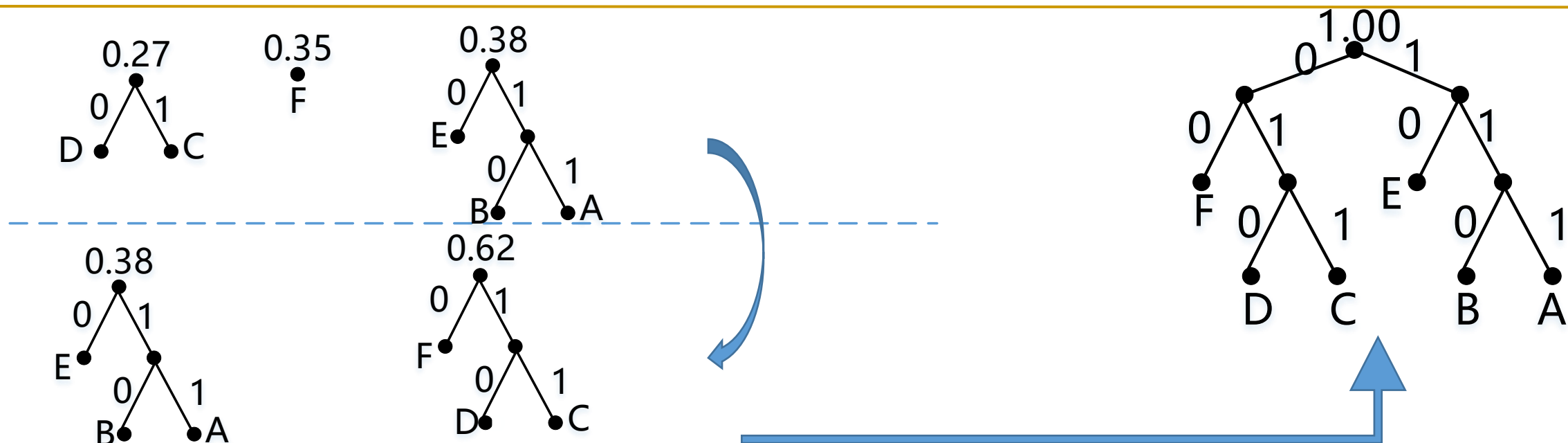
前缀码

□例：用哈夫曼编码来编码下列符号, 其中各个符号的频率如下：A: 0.08, B: 0.10, C:0.12, D:0.15, E:0.20, F:0.35. 求编码一个字符串所需要的平均位数是多少？

□解：根据哈夫曼编码的步骤图如下,



前缀码



□ 所以产生的编码为A是111, B是110, C是011, D是010, E是10, F是00. 使用这种编码来产生编码一个符号所用的平均位数

$$3 \times 0.08 + 3 \times 0.10 + 3 \times 0.12 + 3 \times 0.15 + 2 \times 0.20 + 2 \times 0.35 = 2.45$$

Section 9.3

生成树

内容

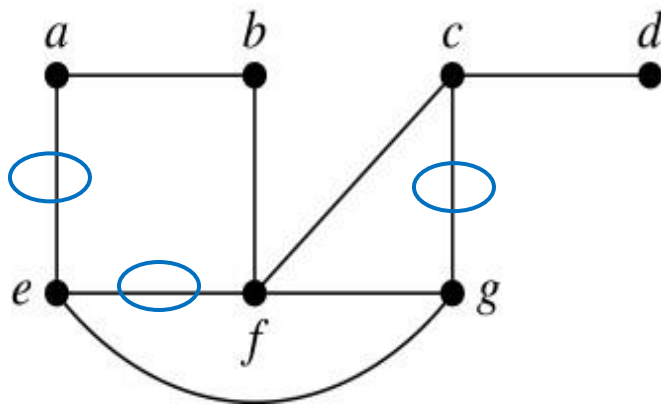
- 生成树
- 深度优先搜索
- 宽度优先搜索
- 回溯的应用
- 有向图中的深度优先搜索

生成树

- 定义: 设 G 是简单图. G 的**生成树**是包含 G 的每个顶点的 G 的子图, 且没有简单回路的连通子图.
- 有生成树的简单图必然是连通的, 因为在生成树中, 任何两个顶点之间都有通路. 反过来也成立, 即每个连通图都有生成树.
- 生成树是原来简单图的所有顶点, 边数最小的, 不包含简单回路的连通子图.
- 图 G 的生成树并不是唯一的.

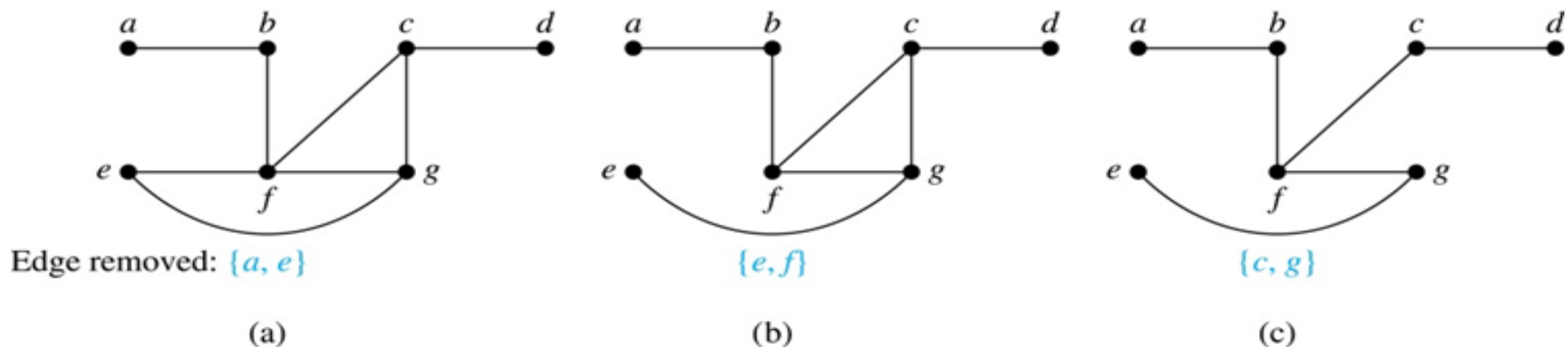
生成树

□例:找出如图所示的简单图G的生成树.

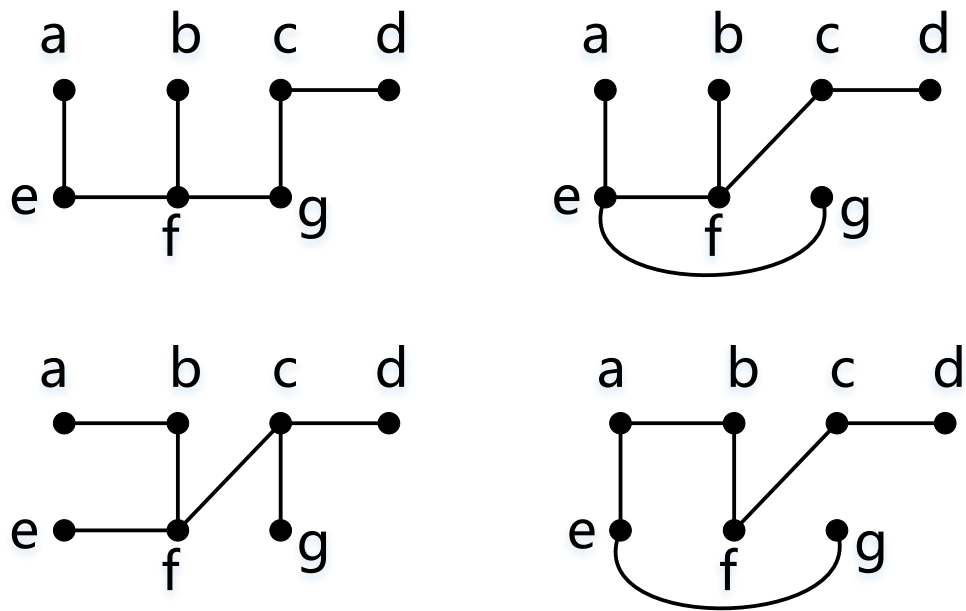


□解:G是连通的, 但不是树, 因为它包含简单回路. 为了找到生成树, 首先删除边 $\{a, e\}$. 这样消除了一个简单回路, 而且得到的子图仍然是连通的且包含G的每个顶点. 然后删除边 $\{e, f\}$, 消除另一个简单回路. 最后删除边 $\{c, g\}$, 产生了一个没有简单回路的简单图, 并且同样包含了G的每个顶点, 所以是一个生成树, 其过程图如下

生成树



另外, G 的生成树不是唯一的, 如下所示的也都是 G 的生成树.



生成树

□定理:简单图是连通的当且仅当它有生成树.

□证:

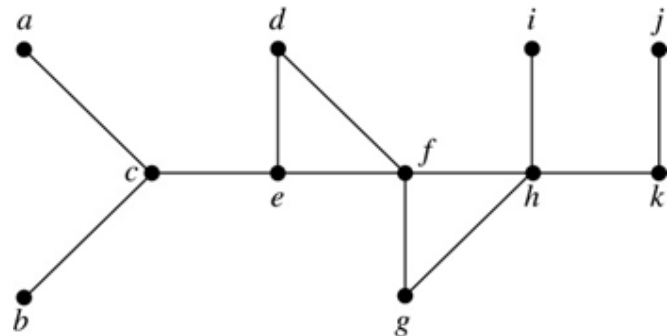
- 假定简单图 G 有生成树 T . T 包含 G 的每个顶点, 且 T 的任何两个顶点之间都有在 T 中的通路. 因为 T 是 G 的子图, 所以 G 的任何两个顶点之间也有通路, 因此 G 一定是连通的.
- 假定 G 是连通的. 若 G 不是树, 则它必定包含简单回路. 从其中删除一条边, 消除掉一个简单回路且保持 G 中的每个顶点都在, 然后再循环这个过程直到没有简单回路为止. 这个过程是可能的, 因为 G 中只有有穷的边数. 当没有简单回路时, 产生一棵树, 因为在删除边的过程时保持了图的连通. 所以这棵树是生成树, 因为它包含了 G 的每个顶点.

深度优先搜索

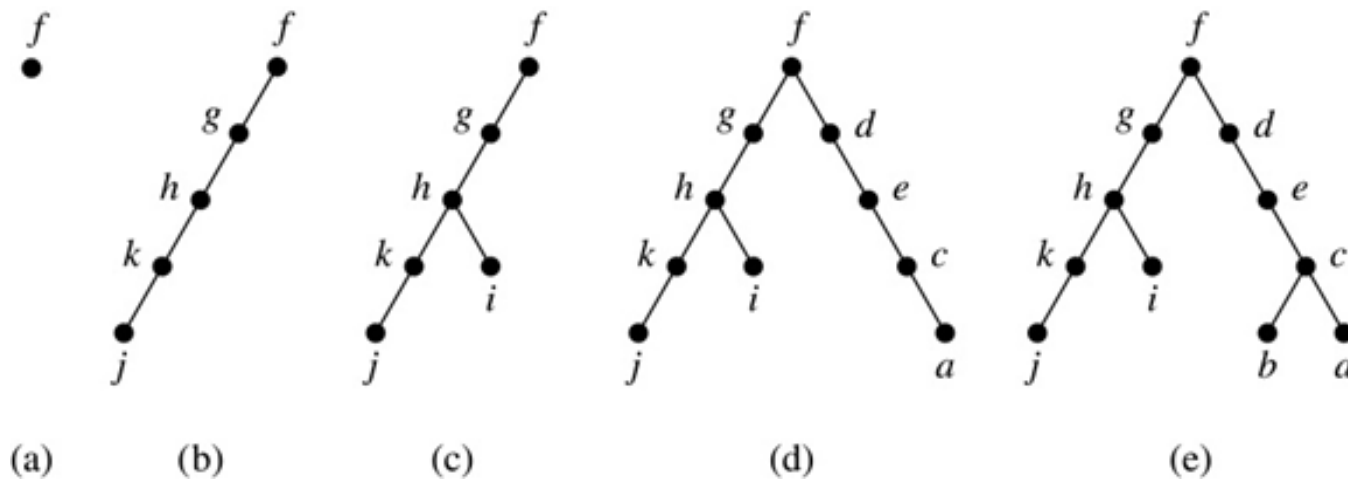
- 前面通过从简单回路删除边来找生成树的过程其实很低效, 因为要找出简单回路. 我们接下来采用另外一种方法来找简单图的生成树:
- **深度优先搜索**建立连通简单图的生成树, 过程如下:
 - 简单图中随机选择一个顶点作为根. 依次添加边来形成从该顶点开始的通路, 其中每条新边都与通路上的最后一个顶点以及还不再通路上的一个顶点相关联. 继续尽可能的添加边到这个通路中.
 - 如果这个通路经过了图的所有顶点, 则这条通路组成的树就是生成树; 如果还没有经过所有顶点, 则必须添加其他顶点和边, 退回到通路中的倒数第二个顶点. 若可能, 则形成从这个顶点开始的经过还没有访问过的顶点的通路; 若不可能, 则后退到通路中的另外一个顶点, 然后再试.
 - 重复以上过程, 从所访问过的最后一个顶点开始, 在通路上一次后退一个顶点, 只要有可能形成新的通路, 直到不能添加更多的边为止.

深度优先搜索

□例: 用深度优先搜索如图所示的简单图G的生成树.

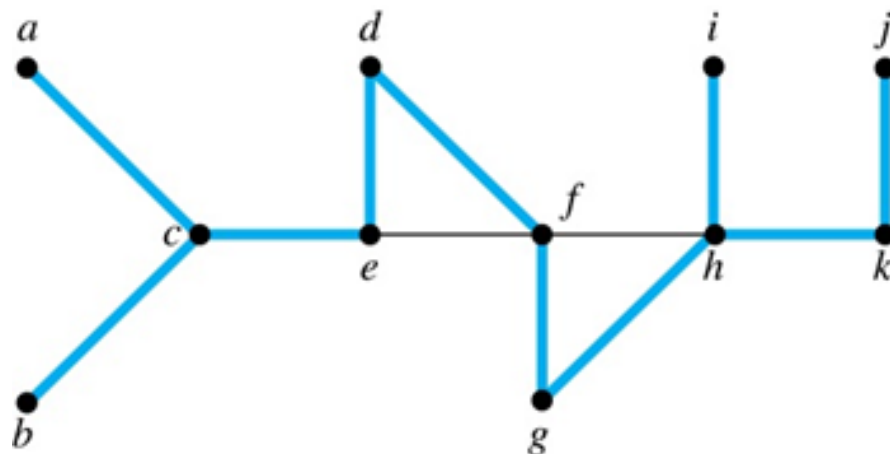


□解:随机选择G中的顶点开始, 如f. 然后将产生如下图所表示的过程的生成树.



深度优先搜索

- 深度优先搜索也称**回溯**, 因为它返回以前访问过的顶点以便添加边.
- 一个图的深度优先搜索选择的边称为**树边**. 这个图所有其他边都必然连接一个顶点与这个顶点在树中的祖先或后代, 这些边称为**背边**.
- 下图显示了上一个例子中深度优先搜索所对应的树边(用粗线显示), 其他背边用细黑线显示:



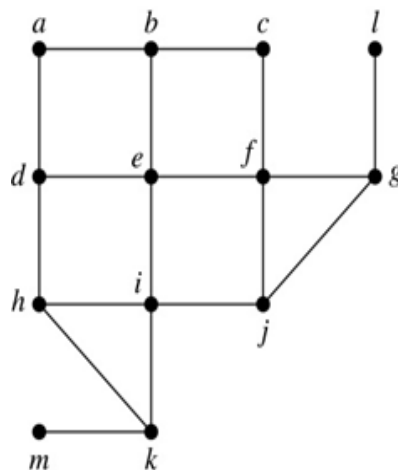
宽度优先搜索

□也可以通过**宽度优先搜索**来产生简单图的生成树, 过程如下:

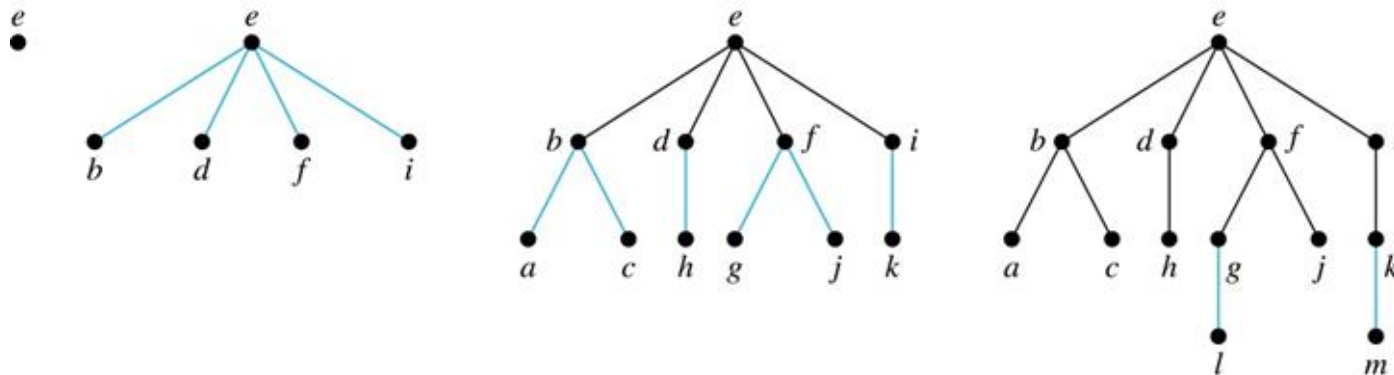
- 简单图中随机选择一个顶点作为根. 依次添加与这个顶点相关联的边. 这个过程中, 所添加的新顶点成为生成树在第1层上的顶点, 将新顶点任意排序.
- 下一步, 顺序访问第1层上的每个顶点, 但不要产生简单回路, 就将与这个顶点关联的边添加到生成树中. 任意排序第一层的每个顶点的孩子.
- 这样就产生了树在第2层上的顶点, 如上循环, 直到已经添加了所有顶点, 对应的生成树也就产生了.

宽度优先搜索

□例: 用宽度优先搜索找出下图的生成树.



□解: 随机选择顶点e作为开始, 其产生的生成树过程如下:

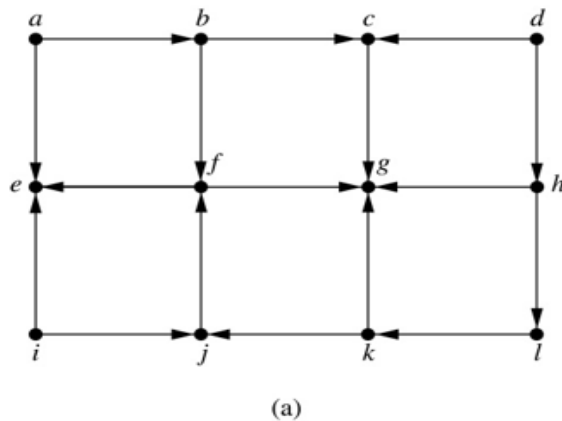


有向图中的深度、宽度优先搜索

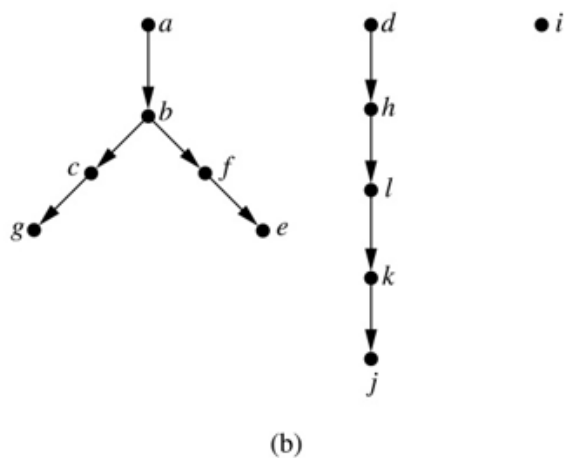
- 可以轻松地修改深度优先搜索、宽度优先搜索, 使得有向图作为输入时他们也能运行. 不过输出不一定是生成树, 而可能是**生成森林**.

有向图中的深度、宽度优先搜索

□例: 如图(a)所示的有向图, 深度优先搜索的输出是什么?



□解: 从顶点a开始深度优先搜索, 输出最后如下所示:



Section 9.4

最小生成树和最优二叉树

9.4.1 内容

- 最小生成树
- 最小生成树算法
 - 普林算法
 - 克鲁斯卡尔算法
- 最优二叉树算法-霍夫曼 (Huffman) 算法

最小生成树算法

- 带权图中, 通过找到一棵使各边的权之和为最小的生成树, 这棵生成树称为**最小生成树**.
- 定义: 连通加权图里的最小生成树是具有边的权之和最小的生成树.

最小生成树算法

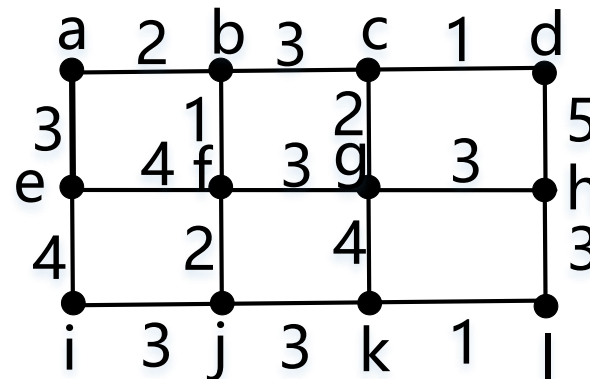
□构造最小生成树的两种经典算法分别是普林算法(**prim**-Jarnik算法)和克鲁斯卡尔算法.

□**普林算法:**

- 首先选择带最小权的边, 把它放进生成树中.
- 依次向生成树中添加与已有树里的顶点关联的, 且不与已在树里的边形成简单回路的权最小的边.
- 当已经添加 $n-1$ 条边时停止.

最小生成树算法

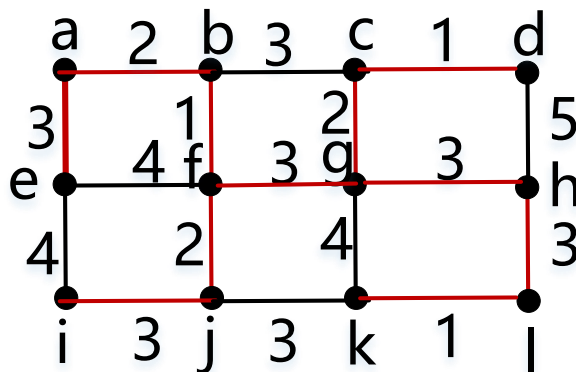
□例1：用普林算法求图示的最小生成树.



□解：

选择	边	权
1	{b,f}	1
2	{a,b}	2
3	{f,j}	2
4	{a,e}	3
5	{i,j}	3
6	{f,g}	3
7	{c,g}	2
8	{c,d}	1
9	{g,h}	3
10	{h,l}	3
11	{k,l}	1

总计:24



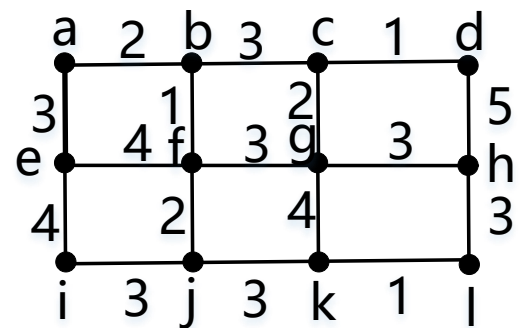
最小生成树算法

□ 克鲁斯卡尔算法 (Kruskal) :

- 首先选择带最小权的边, 把它放进生成树中.
- 依次向生成树中添加不与已经选择的边形成简单回路的权最小的边.
- 当已经挑选 $n-1$ 条边时停止.

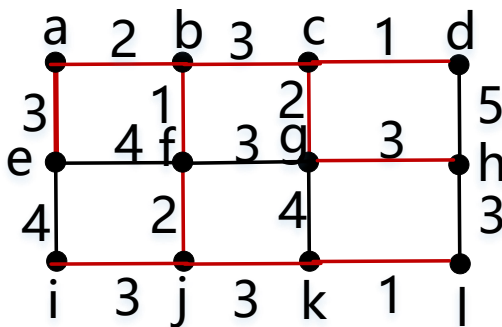
最小生成树算法

□例2：用克鲁斯卡尔算法求图示的最小生成树。



□解：

选择	边	权
1	{c,d}	1
2	{k,l}	1
3	{b,f}	1
4	{c,g}	2
5	{a,b}	2
6	{f,j}	2
7	{b,c}	3
8	{j,k}	3
9	{g,h}	3
10	{i,j}	3
11	{a,e}	3
总计:		24



最优二叉树

□ **定义：** 设二叉树 T 有 t 片树叶 $v_1, v_2, \dots, v_t$, 权分别为 $w_1, w_2, \dots, w_t$, 称 $W(T) = \sum_{i=1}^t w_i l(v_i)$ 为 T 的权, 其中 $l(v_i)$ 是 v_i 的层数. 在所有有 t 片树叶、带权 $w_1, w_2, \dots, w_t$ 的二叉树中, 权最小的二叉树称为最优二叉树.

□ **霍夫曼算法 (Huffman) :**

- 1. 做 t 片树叶, 分别以 $w_1, w_2, \dots, w_t$ 为权.
- 2. 在所有入度为0的顶点 (不一定是树叶) 中选择两个权最小的顶点, 添加一个新的内点, 以这两个顶点作为儿子, 其权等于这两个儿子的权之和.
- 3. 重复步骤2, 直到只有1个入度为0的顶点为止 (根顶点) .
- $W(T)$ 等于所有内点的权之和.

最优二叉树

□例：求带权2,2,3,3,5的最优二叉树.

解：用霍夫曼算法计算最优二叉树的过程. (e) 为最优二叉树, $W(T) = 34$.

